

———— // ZPWR-CLIP-ENGINE // ————

zpwr-clip-engine

v0.1.0

REFERENCE MANUAL

one arranger, every app, one grid – content swapped, never re-implemented

2026-10-01



MENKE
TECHNOLOGIES



MTPUBLISHING



MTAUDIO

Overview

Shared component. "One arranger, every app, one grid – content swapped, never re-implemented." The sequencer/arranger frontend and its scheduling engine were extracted verbatim from `zpw-daw`; hosts mount the grid through a single entry point, `initClipSeq({ nf, getCAT })`, and supply a backend.

`zpw-clip-engine` is the shared DAW arranger / sequencer for the MenkeTechnologies GUI app stack. It is one canvas-2D grid with swappable domains – arranger, notes/piano-roll, launcher/session, the ALS automation timeline, arrangement automation lanes, verified logic clips, and the play-along highway – plus the clip palette, MIDI export, and project/bank serialization. The real scheduling engine lives here too (`engine/`): the pattern → events → step/transport/MIDI scheduler, extracted from `zpw-daw` as a pure-C++17, header-only C ABI (`zpc_clip_*`) – header-only `ClipEngine` + `MidiFile`, no JUCE, no external deps. Created by MenkeTechnologies.

One grid, swappable domains

A single canvas grid renders all eight domains – arranger, piano-roll, launcher/session, the ALS automation timeline, arrangement automation lanes, verified logic clips, the play-along highway and the requests network-run timeline – content is swapped, never re-implemented per app.

One engine, every consumer

The `zpc_clip_*` C ABI builds two ways from one source: `libzpw_clip_engine_static.a` (the daw links it natively) and `libzpw_clip_engine.dylib/.so` (the shared build); the Tauri apps compile the same source into the `bindings/` Rust crate and call the C ABI through it.

Mount through one entry

Hosts call `initClipSeq({ nf, getCAT })` and supply a backend; the `nf` functions bridge the grid to whichever engine the host uses.

No JUCE, no deps

The engine is header-only C++17 (`ClipEngine` + `MidiFile`) with no JUCE and no external dependencies – so it embeds in native and Tauri hosts alike.

Audio & video clips

Beyond MIDI patterns, tracks can hold audio clips (waveform + fades) and video clips (filmstrip + in/out trim). Video clips drop in-browser (frames sampled to a filmstrip), and a program monitor – dockable or popped out into a draggable, resizable player – previews the frame under the playhead. The foundation for a video-editing timeline.

Hosts

- `zpwir-daw` (JUICE / C++) – drives the grid through native `nf` functions backed by this engine plus its own audio voices / per-track instruments (real sound).
- Tauri apps (Audio-Maxor, traderview, ztranslator) – bridge `nf` → the C ABI via a Rust FFI wrapper.
- Domains – the eight modules under `webui/grid/domains/`: arranger, notes / piano-roll, launcher / session, the ALS automation timeline, arrangement automation lanes, verified logic clips, the play-along highway, and the requests network-run timeline.
- Clip palette – the swappable clip bank shared across domains.
- MIDI export – pattern/event data written out via the engine's `MidiFile`.
- Serialization – project / bank save & load.

Mounting

A host mounts the shared grid through one entry point and supplies its backend bridge:

```
import { initClipSeq } from './webui/clip/clip-seq.js';

// nf = the host's bridge functions (native in the daw, FFI in Tauri apps);
// getCAT = the host's content/category provider for the clip palette
initClipSeq({ nf, getCAT });
```

Layout

See the [engineering report](#) for the module map across [webui/](#) (the shared frontend) and [engine/](#) (the C-ABI scheduler).

[MenkeTechnologies](#) · [zpwir-clip-engine](#) · [shared arranger / sequencer + clip engine](#) · [zpwir-clip-engine](#) · [MenkeTechnologiesMeta](#)

getCAT() contract

`clip-seq.js` reads exactly four fields off the catalog; the grid domains never touch it:

field	basic backend	meaning
<code>hasClipSeq</code>	<code>true</code>	a step sequencer exists (drives the clip-view clock)
<code>hasClipAudio</code>	<code>false</code>	native JUCE audio-clip engine wired (daw only)
<code>hasKeyboard</code>	<code>true</code>	playback is allowed (gate on <code>clipStart</code>)
<code>name</code>	<code>'clip'</code>	host id; the daw uses <code>'zpwr-daw'</code> (per-track instruments)

nf(name) contract

`nf(name)` returns a per-command function. `clip-seq.js` wraps almost every call in `try/catch` and persists pattern/arrangement/project state itself via `localStorage` (`zfx_clip_*`, `zfx_clip_arrangement`, ...), so most commands can be neutral resolves. The basic backend never throws and produces no audio.

command	daw (native)	basic backend
<code>clipSeqPattern</code>	loads the pattern into the audio sequencer	no-op resolve (UI persists to <code>localStorage</code>)
<code>clipSeqTransport</code>	sets steps/bpm/div/swing/loop on the engine	real – configures the JS step timer
<code>clipSeqPlay</code>	starts/stops the audio-thread clock	real – starts/stops a <code>setInterval</code> step timer
<code>clipSeqStep</code>	returns the audio clock's current step	real – returns the JS timer's current step
<code>clipSeqExportMidi</code>	writes the SMF via native file IO	real – decodes base64 SMF, browser download
<code>clipSeqSaveFile</code>	native file save (only when <code>window.Juce</code>)	no-op resolve
<code>clipAudioLoad</code>	loads an audio sample, returns peaks/dur	no-op resolve <code>null</code> (never reached)
<code>clipAudioTrigger</code>	fires a native audio voice	no-op resolve (never reached)
<code>clipAudioStopAll</code>	stops all audio voices	no-op resolve (never reached)
<code>setXfade</code>	sets the audio-clip crossfade	no-op resolve
<code>loadTrackInstrument</code>	loads a hosted VST3/AU into a track	no-op resolve
<code>dawSetTrackInstrument</code>	turns a track into a built-in voice synth (empty patch = default note-pitched voice)	no-op resolve
<code>dawClearTrackInstrument</code>	removes a track's built-in voice	no-op resolve
<code>trackPresetSave</code>	writes the active track's graphs to a named <code>.ztrk</code> (<code>{type,note?,stereo?}</code>)	no-op resolve
<code>trackPresetList</code>	lists the saved <code>.ztrk</code> presets (<code>[{name,type}]</code>)	no-op resolve <code>[]</code>
<code>trackPresetLoad</code>	reads a named <code>.ztrk</code> preset back	no-op resolve <code>""</code>
<code>trackPresetDelete</code>	deletes a named <code>.ztrk</code> preset	no-op resolve
<code>getPatch/setPatchJson</code>	reads/writes the track's note graph (preset store)	no-op resolve
<code>trkgetPatch/trksetPatchJson</code>	reads/writes the track's stereo graph (preset store)	no-op resolve
<code>midiNoteOn</code>	native MIDI note-on (audible)	no-op resolve (no synth attached)
<code>midiNoteOff</code>	native MIDI note-off	no-op resolve
<code>setActiveLayer</code>	selects the active synth layer	no-op resolve
<code>setTrackCount</code>	sizes the per-track audio graphs	no-op resolve
<code>clipAutoValue</code>	emits a live automation CC value	no-op resolve

Any command the UI adds later but the table omits falls through to a generic neutral resolver (resolve `undefined`), so the host degrades rather than throws.

Editor commands (clickable and scriptable)

`clip/editor-commands.js` is the one catalog over every editor capability: each registered feature is at minimum a `tool` command, and the media features are also clip-effect commands on the `video` / `audio` / `midi` / `transform` / `automation` / `measure` / `project-tool` surfaces. `commandCount()` is the live total – no count is written into a comment, a placeholder or this README, so adding a feature can never leave a stale number behind. `runCommand(id, surface, ctx)` dispatches; a coverage-invariant test fails if any feature is unreachable as a command.

`clip/clip-command-palette.js` gives that catalog two surfaces, both routed through one `dispatchCommand()` so a click and a script take the identical path:

- The palette. `mountCommandPalette({ container, getEditorContext, onRun })` is a filtered, capped overlay over the whole catalog, bound to OK/Ctrl-K by `initClipSeq`. A media command appends a token to the active clip's effect stack (`effects` / `audioEffects` / `midiEffects`) so it renders through the same paths the buttons use; a tool command computes a value shown in the palette.
- The bus. `publishCommandBus({ getEditorContext, onRun, extraCommands })` registers the same catalog wherever the host has somewhere to put it – as typed verbs on `ZGui.automation` when that bus is loaded, and as rows in the `zpw-r-patch-core` command vocabulary (`window.zpcRegisterCommands`) which feeds `ZGui.palette.setCommands()` and the `zgui:user-command` router. Called at mount, not on first palette open, so the ids resolve even if the user never presses OK.

A verb id is `clip.<surface>.<catalog id>` – structural, never derived from a display label. Two reasons: a label is a translated string, so a label-derived id would rename itself on a locale switch and break every saved chain; and the bare catalog id is not unique (a scalar feature is both a video clip effect and a tool), so the surface is part of the contract.

Each verb declares a reversibility class for the automation bus. A read-only surface (`tool`, `measure`, `project-tool`) only computes, so it is `pure` and runs inside a transaction without being journaled. A media surface with a host `onRun` wired writes an effect onto the clip with no inverse this module can supply, so it declares `irreversible` and a transaction refuses it at call time rather than stranding a chain half-undone at abort.

Rows handed to the host vocabulary are tagged `bulk`, which keeps a four-figure catalog out of the host overlay's row list (it renders every row it is given, and an empty query matches all of them) while leaving every id routable. The browsable surface for the full catalog is this module's own palette, reachable from the host palette as one row.

Ableton Live Set (.als) import

`grid/als-import.js` reads an Ableton Live Set and returns a `serializeProject()`-shaped object, so the host feeds it straight through `deserializeProject` + `applyProjectState` – the same path as a native `.zdp`. This is the cross-DAW interop Bitwig ships: an Ableton arrangement comes in as tracks, clips, MIDI notes, tempo, time signature, arrangement placement, locators, and audio-clip sample paths. Devices/plugins, automation envelopes and audio warping have no target in the clip model and are dropped (as in any cross-DAW import).

`.als` is gzip XML; the host's `gunzipText` decompresses, then routes text starting with `<Ableton` to `alsToProject`. The parser is a dependency-free XML walker (no `DOMParser`, which is absent under Node) so it runs headless in CI. Verified against real sets spanning Live 8.2.1 → Live 12.2.

Float time. A note's exact start is encoded in its fractional grid-step key ("`12.5`" = step $12\frac{1}{2}$); its exact length rides in the optional `bl` field (beats). Integer keys + absent `bl` are the legacy step-sequencer shape, unchanged – old `.zdp` projects load byte-identically. The canvas renderer draws notes at the sub-cell offset and the JS clock micro-delays their note-on. The native step engine (`engine/include/zpc/ClipEngine.h`, the single-clip preview path) has no sub-step slot, so it rounds a fractional start to the nearest step; arrangement playback runs on the float-accurate JS clock.

Verified logic clips (the LOGIC view)

`domains/triggers.js` schedules rule programs as clips: lanes are actions (the host injects them – `clip-seq.js` uses tracks), cells are time slots, and each painted cell carries `{ source, rules }`, a ztranslator `Rule` program.

The host verifies each program before it can be armed. `clip-seq.js` debounces 120 ms per keystroke and calls `ztInvoke("ztr_verify_source", { text, label })`, which parses and runs `verify::analyze_program` in one round trip. Arming is refused while the analysis reports any `Severity::Error` finding – dead code, contradiction, division by zero, infinite loop, broken goto. Warnings and infos render in the findings table but do not block. A parse failure comes back as `{ error }` and is never rendered as “0 findings”.

`serialize` emits `[{ slot, action, program }]` sorted by `(slot, action)` so a save is diff-stable; `deserialize` stays permanently tolerant of the older flat `{ slot, action, payload }` shape, so grids saved before logic clips existed still load (the cell paints and stays unarmed).

Engine side: `ClipEvent::Logic = 2` is additive over `NoteOn = 0 / NoteOff = 1`. `setLogicClips / zpc_clip_set_logic` install a `[{ step, logicId }]` table that fires on the same swung step boundary as notes – one timebase, so logic never drifts against the notes it accompanies. An empty table restores note-only behaviour exactly. The rules VM runs host-side, never on the audio thread.

The requests domain (a network run on the arrangement grid)

`domains/requests.js` is the proof that the grid is not an audio widget: it lays a run of HTTP requests on the same canvas, with real milliseconds on the time axis instead of bars. It ships two factories over one shared ruler, the same way `arranger` and `autolanes` pair up:

- `runAxis({ getTotalMs, getUnitMs })` – the ruler. Milliseconds become fixed ticks grouped into whole seconds. Both domains take the same instance, so a change of tick size can never drift the two grids apart.
- `createRequestsDomain({ axis, getLanes, getSends, getMarkers? })` – lanes are the requests, and each send is a `clip` region whose left edge is `at_ms` and whose width is `time_ms`. `fromRun(items, laneOf)` builds the model shape straight from a run payload. Regions are colour-coded by outcome class (`SEND_CLASSES`: 2xx / 3xx / 4xx / 5xx / no-reply, where a transport failure is its own class rather than a fake 5xx).
- `createRequestMetricsDomain({ axis, getSends, getBackoff? })` – three breakpoint-envelope lanes on the identical axis: latency (sampled where each send completed), status, and retry backoff (the host's own retry planner supplies `[{ at_ms, delay_ms }]`). `denorm(laneId, v)` returns a normalized point in the lane's own unit.

A cell's `clipId` is an index into `getSends()`, not an inline payload. That is load-bearing rather than stylistic: `grid-interactions` rebuilds a resized `clip` value as exactly `{ clipId, len }` and drops every other field, so any per-region data has to be reachable from the id alone or a resize erases it.

The host pairs this with the grid's own loop ruler: the braces select a window of the axis, and re-firing that window re-sends exactly the requests whose regions overlap it. `zreq` drives it that way (`runner.loop`).

The highway domain (play-along)

`domains/highway.js` is a surface over the same `ClipSeq` pattern the notes and arranger domains edit, so a chart is just a pattern: anything authored in the editor can be played along to, and anything imported to play along to opens in the editor. There is no second note format and no converter.

Editing is off by default – a stray drag during a take would silently rewrite the chart being graded against. A host that wants the chart editable between takes passes `editable`, either a flag or a predicate:

```
createHighwayDomain({ mode: 'drums', lanes, editable: () => !takeRunning })
```

`capabilities` is then a live read (`grid-interactions.js` asks per gesture), so the same grid is a frozen play surface while a take runs and a full FL editor the moment it stops – no rebuilt grid, no second canvas.

Three lane surfaces cover the EDM controllers: `keyLanes(lo, hi)` (pitch lanes, high note on top, every lane named with its pitch and octave – accidentals included, which is why the keys gutter is wider than a two-character label needs), `MPC_PAD_LANES` (4×4, pad 1 = note 36 = bottom-left), and `GM_DRUM_LANES` (named GM percussion, kick at the bottom). `unmappedNotes` reports which chart notes the chosen surface cannot show, so an import never presents a chart as playable while part of it is invisible.

A lane carrying no label (charts serialized before every key lane was named) falls back to its pitch name at render on pitch surfaces; pads and drums keep their piece names. A note head is clamped to the note's own half-width, and where that is too small to write in, the name is drawn beside the note. The optional `litLanes()` dep (a Set or array of lane ids or pitches, asked per frame) lights the gutter rows that are sounding in the playhead colour.

`createHighwayScroller` parks the playhead on a fixed judgment line: each frame it converts the transport position to a fractional step via `unitAtTime` and scrolls so notes arrive at the line exactly when due. `unitAtTime` interpolates a `timeOfStep(0..steps)` table fetched once per transport change, which keeps the render loop free of per-frame engine IPC while still landing on the engine's own swung boundaries – dividing elapsed time by an average step length puts notes on the wrong side of the line under any swing.

A count-in runs the clock negative so every target keeps its authored time; `unitAtTime` extrapolates negative time backwards at the first step's rate, so the chart starts to the right of the judgment line and walks in, arriving as the count reaches zero. `setPlayhead(null)` clears the playhead (`-1` is still accepted as clear), so a negative position during the count-in still draws the line.

The domain does not grade. Grading compares an input timestamp to a target time and belongs wherever the audio/MIDI clock lives; a second implementation here would drift from it and return different verdicts for the same performance. The highway renders verdicts it is handed.

The judgment line needs empty space before the first note and past the last, so the domain declares `overscrollUnits` on its time axis – optional runway that `computeLayout` adds to the horizontal scroll range. Domains that do not declare it get `0`, i.e. exactly the previous clamp.

The FL tool set

`fl-tools.js` is the tool table every grid dispatches on, and the pure maths the tools need (`sliceSpan`, `zoomRangeFor`, `muteToggled`, `slipped`) – kept out of the event code so the behaviour is unit-testable in node. Each tool has FL's own letter shortcut, which `grid-interactions.js` handles when the host passes `setTool`:

Tool	Key	Left-drag does
Draw	P	place · drag to move · edge to resize · Alt-drag duplicates
Paint	B	paint many across time and lanes
Paint drums	N	paint; click an existing item mutes/unmutes it
Delete	D	click or sweep to remove
Mute	T	click or sweep to mute/unmute (notes and clips)
Interpolate	I	ramp the values between press and release
Slice	C	cut every item the line crosses, at the crossing unit
Select	E	click or marquee, then Delete / bulk edit
Zoom	Z	fit the dragged range; a click on the background zooms out
Play	Y	audition what is clicked; drag scrubs
Slip	S	slide a clip's content, keeping its start and end

A tool the domain cannot use falls back to Draw rather than leaving an inert pointer (`effectiveTool`), and every tool draws its own cursor from `cursors.js` – in FL the cursor is how you know which tool has the mouse.

Tools are also split by whether they WRITE (`toolMutates`). A domain with every capability off is a display, and the mutating tools – plus the keyboard edits (paste, nudge, transpose, delete-selection) – are refused on it outright, while Select, Zoom and Play still answer. That gate is what makes `editable: () => ...` trustworthy: without it a left-click still placed a note on a “read-only” surface.

Hosts wire it either way: `createGrid` holds the tool itself (pass `getTool` / `setTool` to own it instead) and exposes `grid.setTool(id)`; `clip-seq.js` drives its toolbar buttons straight off `FL_TOOLS`, so a tool added to the table appears in the UI with no second list to update. Two optional deps make the last two tools whole: `zoomToRange(lo, hi)` (the renderer supplies one) and `onAudition(info)`, since only the host owns a voice.

Timing queries (play-along scoring)

`advance` + `poll_events` tell a host that a step fired, not when it was due. A play-along grader needs the second thing: an input timestamp is only meaningful against the time the chart expected the note. Three queries expose the engine's own clock so a host never re-derives the swing math (a re-derivation that drifts from what is playing produces wrong grades silently, with no crash and no error):

C ABI	C++	Meaning
<code>zpc_clip_position_seconds</code>	<code>positionSeconds()</code>	Seconds since <code>play(1)</code> , monotonic across loop wraps; 0 while stopped.
<code>zpc_clip_time_of_step</code>	<code>timeOfStep(step)</code>	Seconds from the top of a pass to the top of <code>step</code> within that pass.
<code>zpc_clip_pass_duration_seconds</code>	<code>passDurationSeconds()</code>	Seconds per full pass – how much <code>positionSeconds</code> grows per wrap.

`timeOfStep` is pass-relative, not absolute. A loop wrap resets the step to 0 (`advanceStep`), which restarts the swing phase, so a continuous step index is only correct when the pattern length is a whole number of swing cycles. Convert a chart target to the `positionSeconds` timeline with:

```
targetSeconds = pass * passDurationSeconds() + timeOfStep(stepWithinPass)
```

With a 6-step pattern at `swingUnit = 4` (an 8-step cycle), treating the index as continuous is already 47 ms off in pass 1 and 141 ms off in pass 2, growing every pass. `engine/tests/timing_test.cpp` pins the invariant that catches this: at the moment step `k` of pass `p` fires, `positionSeconds()` equals the formula above.

Architecture

zpw-clip-engine is consumed two ways from one source. The frontend (`webui/`) is mounted by hosts through `initClipSeq({ nf, getCAT })`: one canvas grid renders swappable domains (arranger, notes/piano-roll, launcher/session, the ALS automation timeline, arrangement automation lanes, verified logic clips, the play-along highway, and the requests network-run timeline) plus the clip palette. The engine (`engine/`) is the scheduler as a pure-C++17, header-only C ABI (`zpc_clip_*`) – `ClipEngine` + `MidiFile`, no JUCE, no external deps – built both as a static lib the JUCE daw links natively and as a `dylib/.so`; the Tauri apps compile the same source into the `bindings/` Rust crate and call the C ABI through it. The `nf` bridge is what differs per host; the grid and the scheduler do not.

Module map

area	responsibility
<code>webui/grid</code>	The canvas grid – model, interactions, cursors, sequencer, project state, domain entry (<code>grid-core.js</code> , <code>grid-model.js</code> , <code>grid-interactions.js</code> , <code>sequencer.js</code> , <code>project.js</code> , <code>als-import.js</code>).
<code>webui/clip</code>	The clip layer – clip grid, clip bank/palette, sequencer view (<code>clip-seq.js</code> , the host entry point), and the non-audio basic backend for preview (<code>clip.js</code> , <code>clip-grid.js</code> , <code>clip-bank.js</code> , <code>clip-basic-backend.js</code>).
<code>webui/lib</code>	Submodule mount point: <code>webui/lib/zgui-core</code> is the shared zgui-core component library (see <code>.gitmodules</code>), not source of this repo.
<code>engine/</code>	The C++17 header-only scheduler exposed as the <code>zpc_clip_*</code> C ABI – <code>ClipEngine</code> (pattern → events → step/transport) and <code>MidiFile</code> (SMF write). Builds as a static lib and a shared lib from one source.
<code>bindings/</code>	The Rust FFI crate the Tauri apps depend on: <code>build.rs</code> compiles <code>engine/src/capi/clip_engine.cpp</code> in via <code>cc</code> , and <code>src/lib.rs</code> wraps the C ABI as <code>#[tauri::command]</code> s, bridging <code>nf</code> calls onto the C ABI.